~/majibill/docs/01-system-architecture.md

$ cat 01-system-architecture.md

MAJIBILL PLATFORM · ARCHITECTURE

System Architecture & Design

Control plane, cells, modular monolith, and the payment path — how MajiBill is shaped to serve a whole county and still be sold as a single-client system.

```
tenant_id = current_setting('app.tenant_id')::uuid
-- row level security: ON
-- every write is audited, every payment is idempotent
```

DOC	VERSION	DATE
MB-01	1.0 draft	08 Oct 2026

STATUS	AUDIENCE	CLASS
For review	Engineering leads, architects, technical reviewers	Confidential

Prepared by 404HubSpot · Nakuru, Kenya · https://404-hub-spot.vercel.app

Working codename: MajiBill (Swahili: maji = water). Figures marked 'assumption' must be validated before commitment.

// Contents

1. Purpose and how to read this . . . 3
2. System context . . . 4
3. Control plane and data plane (cells) . . . 6
4. Logical architecture . . . 8
5. Monolith or microservices? The decision . . . 10
6. Runtime flows . . . 11
7. Data architecture overview . . . 13
8. Non-functional targets . . . 14

// 1. Purpose and how to read this

This document defines the shape of the system: who the tenants are, how they are isolated, which parts are deployed separately, and how the money path stays correct under load. Backend internals, schema, infrastructure and audit have their own documents; this one is the map.

Question	Short answer
Multitenant or single-client?	One codebase, **multitenant-native**, deployable three ways: pooled cell, dedicated (silo) cell, or client-owned (sovereign) install.
Microservices?	No. A **modular monolith** core plus three small, purpose-built satellites (payment ingress, worker pool, analytics). Extraction is trigger-based, not fashion-based.
Who owns the platform?	404HubSpot owns the IP and runs the control plane. Partners (ZUCAR) resell; tenants (utilities, estates) own their data.
Scale target?	A Nairobi-scale utility (assumption: 500k–750k active accounts) on a dedicated cell, without redesign.

1.1 Architecture principles

1. **Tenant isolation is a platform invariant**, enforced below application code (Postgres row-level security), never left to developer discipline.
2. **Money is a ledger, not a column.** Balances are derived from append-only entries; corrections are reversals, never edits.
3. **Every edge is idempotent.** Payments, readings, SMS and webhooks can be retried safely; duplicates are impossible by constraint.
4. **Boring technology in the core,** specialised languages only where the workload justifies them (see MB-02).
5. **Offline-first in the field.** Meter readers work without signal; the server is the arbiter on sync.
6. **Configuration over code** for tariffs, charges, penalties, templates and workflows; a new utility is data, not a fork.
7. **Audit and observability by default,** not bolted on (see MB-06).
8. **Cells for growth.** Capacity and blast radius scale by adding cells, not by growing one giant cluster.
9. **Portable.** Containers + Terraform; no hard dependency on a single cloud's proprietary services in the core, so a sovereign install is possible.

// 2. System context

MajiBill sits between water customers and the utility's revenue process. It talks to Safaricom (M-Pesa), an SMS gateway and email provider today; banks, cards and smart meters are planned integrations (dashed).

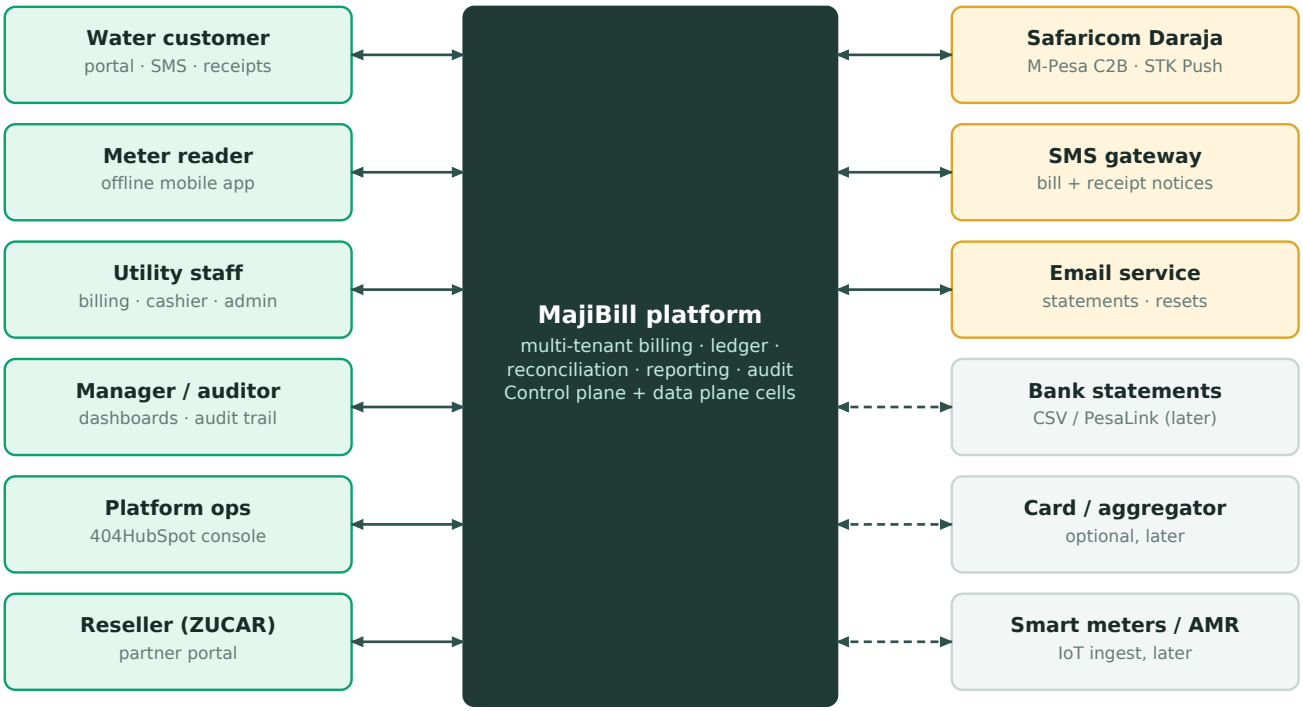


Fig 2.1 System context: actors (left), external systems (right). Dashed = later phases.

2.1 Who is the tenant?

In Kenya, water services are delivered by licensed Water Service Providers (WSPs) regulated by WASREB; its sector reporting covers roughly 90+ WSPs across the 47 counties. A **county is therefore not automatically the tenant**: the tenant is the operating organisation (a WSP, an estate manager, an institution). ZUCAR is modelled as a **reseller partner** that may manage several tenants. If ZUCAR also operates a water service itself, it simply has a tenant of its own.

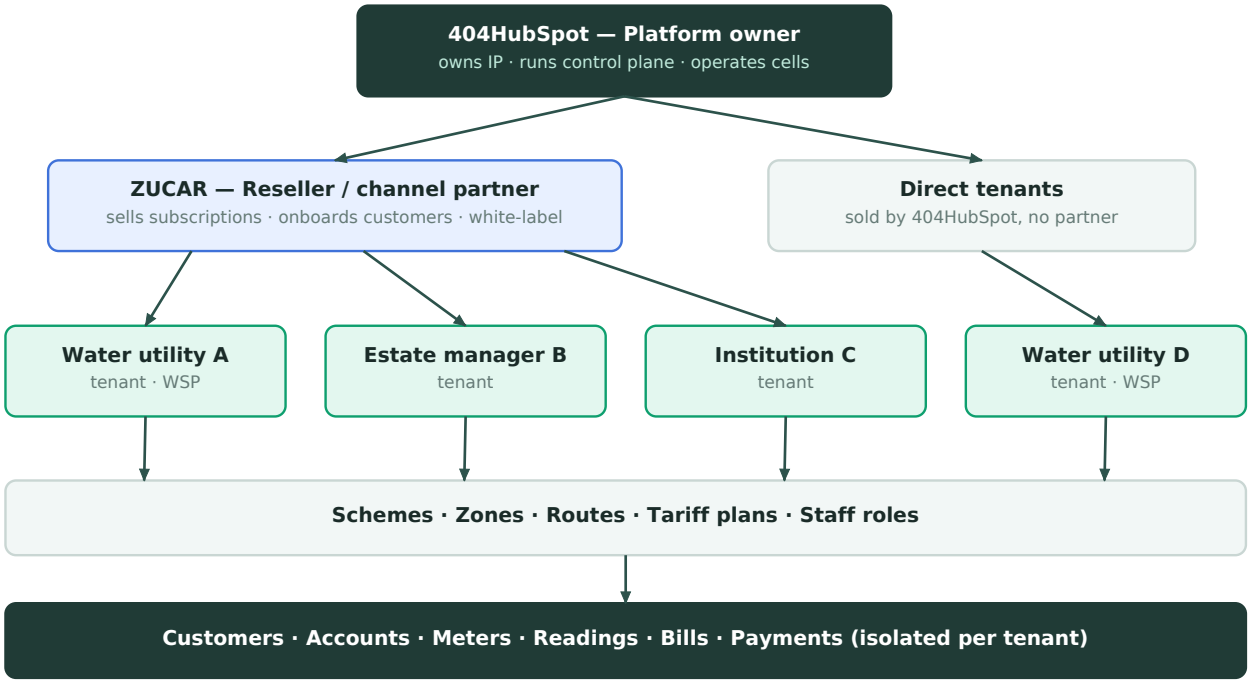


Fig 2.2 Business hierarchy: platform owner → partner → tenant → operational units → customers.

Entity	Meaning	Owns data?
Platform	404HubSpot. Runs control plane, cells, releases, support.	Platform metadata only
Partner	Reseller such as ZUCAR. Sells, onboards, brands; sees aggregated tenant health, never raw customer PII unless granted.	No (by default)
Tenant	A utility / estate / institution. Unit of isolation, billing, config, retention and export.	**Yes**
Scheme / zone / route	Operational partition inside a tenant: supply scheme, meter-reading route, collection zone.	Tenant
Account / meter / customer	Billing account, physical meter, and the person or organisation paying.	Tenant

// 3. Control plane and data plane (cells)

The control plane is how 404HubSpot **runs the business of the platform**: it knows every tenant, which cell hosts it, which plan it is on, and what it is entitled to. The data plane is where tenant data and billing run. Splitting them early is the single biggest lever for later scale, enterprise silos, and the “client wants to buy it” scenario.

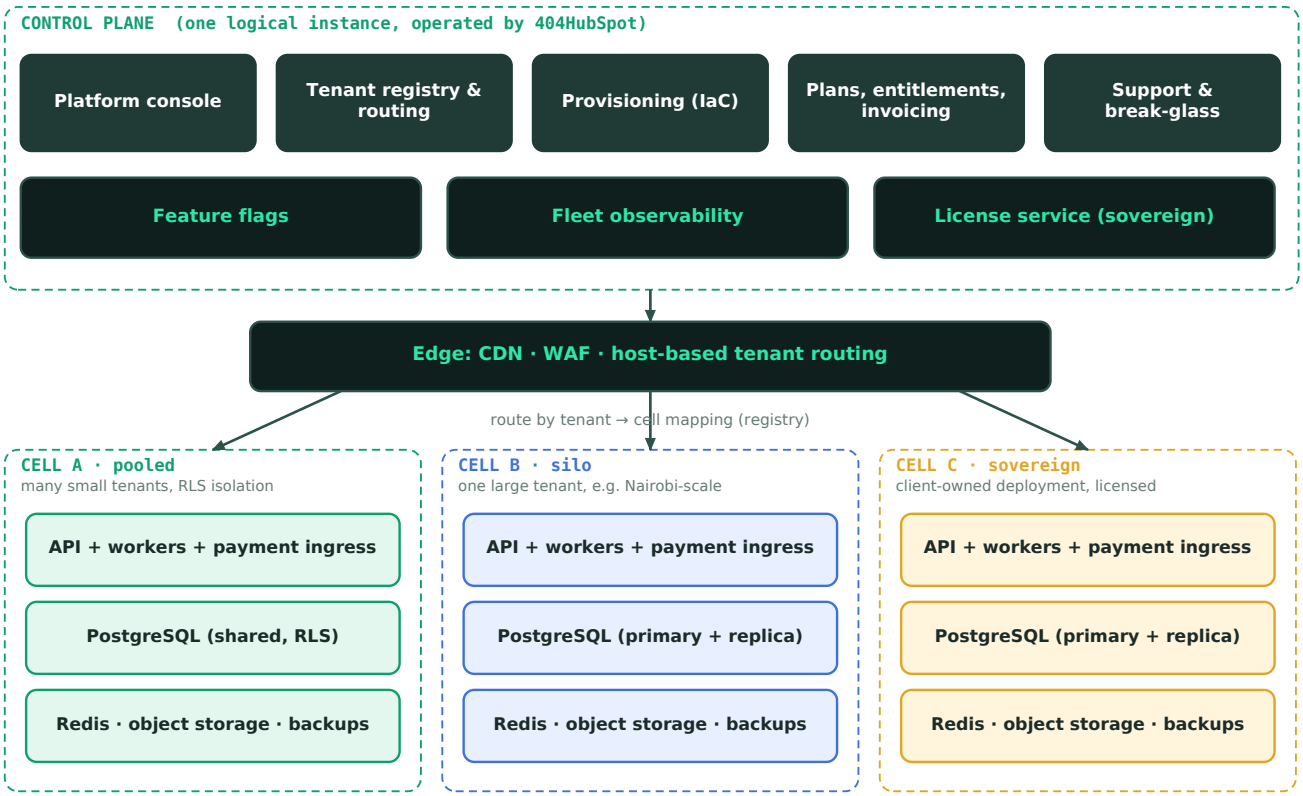


Fig 3.1 Control plane above; data-plane cells below. A cell is a complete, independently deployable copy of the stack.

Cell type	Hosts	Isolation	Typical use
A — pooled	Many small/medium tenants	Shared DB, `tenant_id` + RLS, per-tenant rate limits and quotas	Estates, small WSPs, pilots
B — silo	One large tenant	Dedicated DB, workers, queues, KMS keys; same artifact	Nairobi-scale utility, regulated or contractual isolation
C — sovereign	Client-owned install	Client’s cloud/on-prem; optional license service for updates	“We want to buy the system”

DECISION **Pooled by default, silo on demand, one artifact**

All three cell types run the **same container images and migrations**. A tenant moves from A to B with a controlled data move and a registry change, not a rewrite. This resolves the “multitenant vs sellable single system” question without maintaining two products.

3.1 Isolation models compared

Model	Pros	Cons	Verdict
Shared schema + RLS (pooled)	Cheapest, simplest ops, one migration path, fast onboarding	Noisy neighbour risk; one bug class (missing tenant context) is catastrophic unless enforced by RLS	**Default**
Schema per tenant	Clean logical split	Migration fan-out, connection/catalog bloat, awkward cross-tenant reporting for partners	Rejected
Database per tenant (silo)	Strongest isolation, per-tenant restore, easy export/sale	Higher cost and ops per tenant	**Enterprise tier**

> WATCH OUT **RLS must be tested like a security feature**
> Every tenant-owned table gets a policy; CI fails if a table with a `tenant_id` has no policy. The app role is non-superuser and cannot `BYPASSRLS`. See MB-03 and MB-07.

// 4. Logical architecture

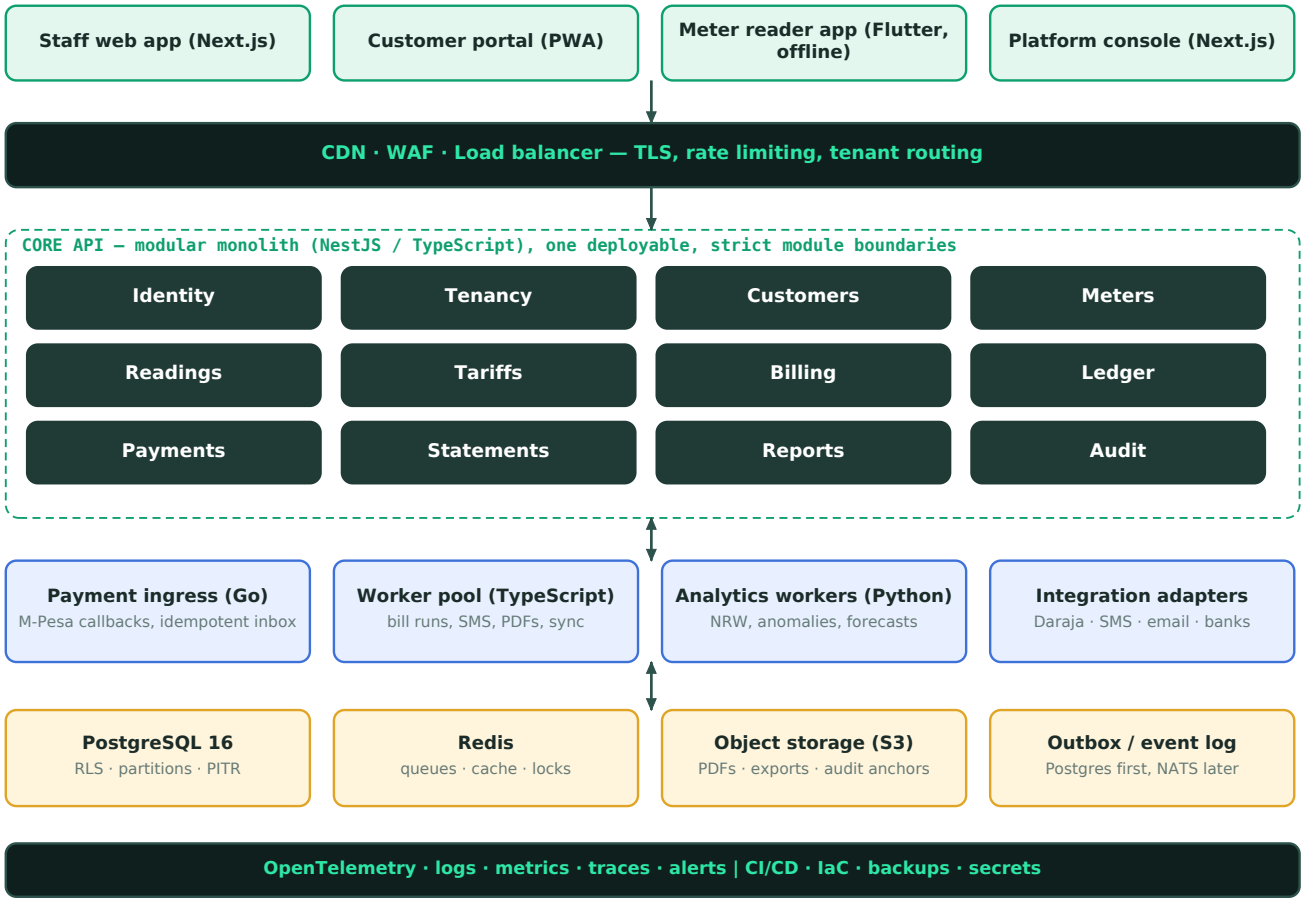


Fig 4.1 Layered view: clients → edge → core modular monolith → satellites → data → observability.

4.1 Deployables

Deployable	Language	Why this language	Scales by
`core-api`	TypeScript (NestJS)	Largest surface area; shared contracts with web; strong typing for domain code; huge hiring/agent leverage	Horizontal replicas
`workers`	TypeScript (same repo, different entrypoint)	Reuses domain modules for bill runs, notifications, PDFs, sync processing	Queue depth
`payment-ingress`	Go	Tiny, stateless, high-concurrency callback receiver with fast, predictable acks	Horizontal replicas
`analytics`	Python	Anomaly detection, NRW analysis, forecasting, data migration/ETL (pandas/SQL)	Batch jobs
`web`, `console`	TypeScript (Next.js, React)	Staff app, customer portal, platform console	CDN + replicas
`reader-app`	Dart (Flutter)	Offline SQLite, camera, background sync on low-end Android	Per release

Rust is deliberately not in v1. Candidate later use: a deterministic tariff engine compiled once and shared by server and offline mobile estimates, if profiling or correctness requirements justify it (MB-02, ADR-009).

4.2 Core module map

Module	Owns	Publishes events
identity	Users, sessions, MFA, API tokens, RBAC grants	user.created, login.failed
tenancy	Tenants, partners, plans, settings, branding, tenant context	tenant.provisioned
customers	Customers, accounts, contacts, status lifecycle	account.opened, account.closed
meters	Meters, installs, replacements, routes, zones	meter.replaced
readings	Reading capture, validation, anomalies, estimates	reading.accepted, reading.flagged
tariffs	Tariff plans, block rates, fixed charges, penalties, effective dating	tariff.published
billing	Cycles, bill runs, bills, adjustments, approvals	bill.issued, bill.adjusted
ledger	Immutable entries, allocation, balances, write-offs	ledger.posted
payments	Inbox, matching, suspense, receipts, refunds, reconciliation	payment.received, payment.unmatched
statements / reports	Statements, WASREB-aligned KPIs, exports, read models	report.generated
audit	Append-only audit events, hash chain, anchors, viewer	—
notifications	SMS/email templates, sending, delivery receipts, wallet	notification.sent

Rule: modules talk through public interfaces and events only; no module reads another module’s tables. Enforced with lint boundary rules and a CI import-graph test.

// 5. Monolith or microservices? The decision

Assumed team: 5–8 engineers, agent-assisted, startup stage, one client in sight, one production environment, regulated money flow. Evaluate three options honestly:

Criterion	Full microservices	**Modular monolith + 3 satellites**	Unstructured monolith
Time to first pilot	Slow (platform tax up front)	**Fast**	Fastest, then slows
Operational load for the team	High (many services, tracing, contracts)	**Moderate**	Low
Money-path consistency	Hard (distributed transactions, sagas everywhere)	**Strong** (single ACID database for ledger)	Strong
Future extraction	Already done	**Clean seams, trigger-based**	Painful
Language fit per workload	Excellent	**Good where it matters**	Poor
Sovereign / on-prem install	Heavy to ship	**One compose file or small chart**	Easy
Failure blast radius	Small per service	Core is one unit; satellites isolate the spiky parts	Everything

> DECISION **Decision**
> Modular monolith core with three satellites split for **workload shape**, not for fashion: payment ingress (spiky, must ack fast, tiny), workers (long-running, queue-driven), analytics (different language and runtime). Extract more only on evidence.

Extraction triggers (any one, sustained)

- A module needs an independent release cadence from at least two teams.
- A module's load profile forces scaling that wastes capacity if kept in the core (e.g. reporting, PDF rendering).
- A module needs a different runtime or compliance boundary (e.g. payment data scope).
- Deploy coupling causes repeated release delays or incidents.

// 6. Runtime flows

6.1 Tenant request path

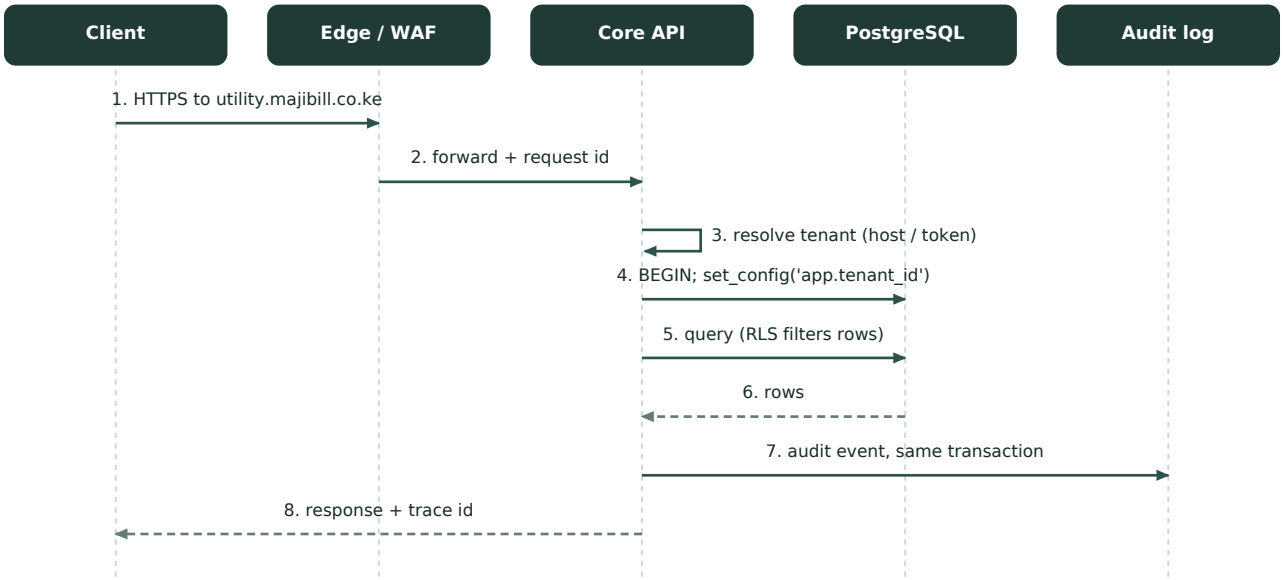


Fig 6.1 Tenant context is set per transaction; RLS makes cross-tenant reads impossible.

6.2 Reading to bill (offline-first)

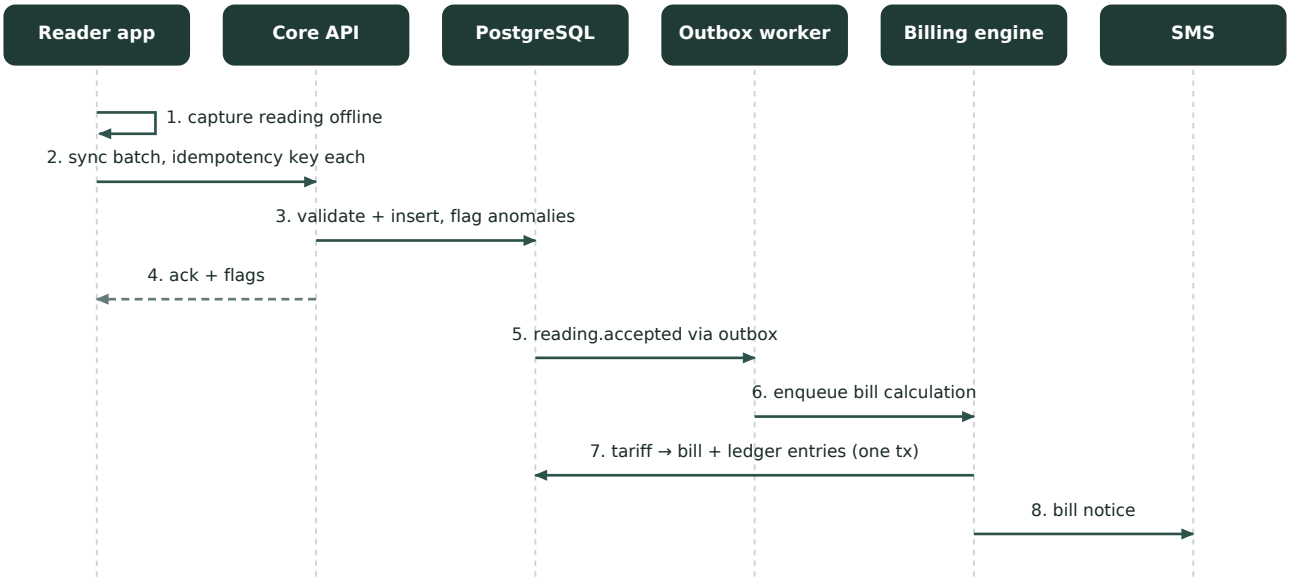


Fig 6.2 A reading is accepted once, billed once, and notified once, even if the phone retries.

6.3 M-Pesa payment (the critical path)

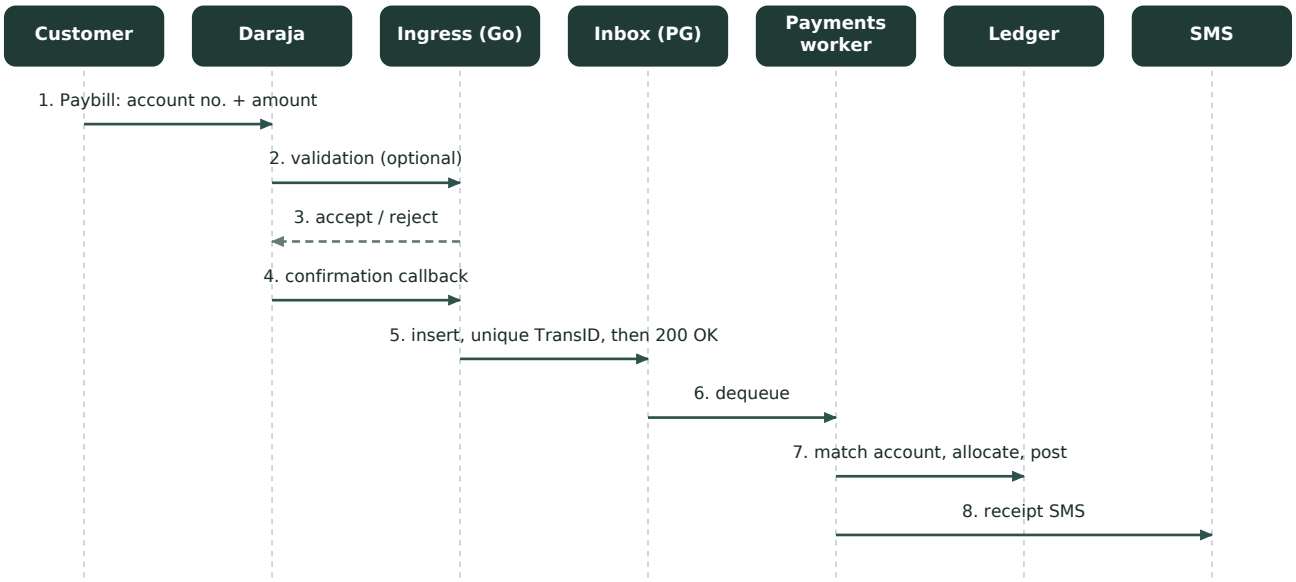


Fig 6.3 The ingress service does one thing: persist the callback durably and acknowledge fast. Business logic runs after.

- Unique constraint on `(tenant_id, provider, provider_txn_id)` makes replays harmless.
- Unmatched or ambiguous payments go to a **suspense account** with an operator queue; nothing is dropped.
- A scheduled **reconciliation job** compares ingested payments to provider statements / transaction-status queries and raises gaps.
- Each tenant uses **its own Paybill/Till** so funds settle directly to the utility; the platform never holds customer money (to be confirmed with Safaricom and counsel, see MB-07).

6.4 Monthly bill run pipeline

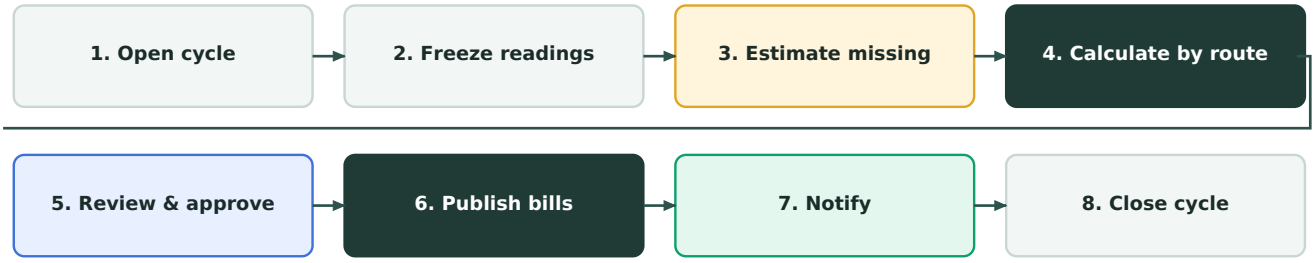


Fig 6.4 Bill run: parallel by route, idempotent per account-cycle, resumable after failure. Step 5 is a maker–checker gate.

Each account-cycle calculation is keyed `(tenant, account, cycle)` so a restarted run never double-bills. Workers process routes in parallel; progress and failures are visible in the console; a failed account does not block the rest.

// 7. Data architecture overview

Concern	Choice	Notes
System of record	PostgreSQL 16+	Single ACID store for readings, bills, ledger, payments. RLS for tenancy. Monthly partitions on big append-only tables.
Cache / queues / locks	Redis	Job queues (BullMQ), rate limits, short-lived state. Never a source of truth.
Events	Transactional outbox in Postgres	Reliable publish-after-commit. A broker (NATS JetStream) is an option when consumers multiply.
Files	S3-compatible object storage	Bills, statements, exports, photo evidence, audit anchors (object lock).
Reporting	Read replica + materialised views; columnar store later	Heavy partner/WASREB reporting must not compete with billing.
Search	Postgres trigram / full-text	Customer lookup by name, phone, account, meter. A search engine only if proven necessary.

7.1 Consistency, idempotency and failure handling

Hazard	Mechanism
Duplicate payment callback	Unique `(tenant, provider, txn_id)`; insert-or-ignore; replay returns same result
Reading retried by flaky mobile network	Client-generated UUID per reading; unique constraint; ack returns existing row
Crash mid bill run	Per account-cycle idempotency key; run state machine; resume, never restart
Lost event after commit	Outbox row written in the same transaction; relay retries until acked
Poison message	Bounded retries with backoff, then dead-letter queue with operator alert
Wrong balance	Balance = sum of ledger entries; nightly invariant check (debits = credits per tenant)
Clock skew on devices	Server timestamps are authoritative; device time stored but never trusted for ordering

7.2 Capacity model (Nairobi-scale assumption)

Population does not size the system; **active billable connections** do. The figures below are planning assumptions to be replaced with the real utility's numbers during discovery.

Quantity	Assumption	Derived
Active accounts	750,000 (upper planning case)	—
Bills per month	1 per account	750k bills, 9M per year
Ledger entries per bill cycle	about 4 (charge, fixed, arrears carry, penalty)	about 36M per year
Payments per month	about 80% of accounts, some paying twice	about 600k, about 7M per year
Bill run throughput needed	complete in under 2 hours	about 105 bills/second sustained
Payment peak	8% of monthly volume on the busiest day, 15% of that in the busiest hour	about 2 tx/s average, design for 100 tx/s burst
Audit events	about 10 per account per month	about 90M per year (partitioned, compressible)
Primary DB size	rows + indexes, first year	roughly 150–300 GB (to be validated by load test)

> NOTE **What this tells us**
> This is a **correctness and operations** problem far more than a raw-throughput problem. One well-run Postgres primary with a replica handles it comfortably; the effort goes into idempotency, reconciliation, backups, and observability.

// 8. Non-functional targets

Attribute	Target (v1)	How we get there
Availability	99.9% monthly for tenant-facing API (about 43 min budget)	Multi-AZ DB, 2+ API replicas, health-checked rollouts
Latency	p95 under 400 ms for staff app reads; under 1 s writes	Indexes by tenant-first keys, caching, read replica
Payment visibility	Payment shows on account within 60 s of confirmation	Ingress inbox + worker autoscaling, queue-lag alert
Bill run	750k bills in under 2 h	Route-parallel workers, batch writes, partitioned tables
Recovery	RPO 5 min, RTO 1 h (enterprise); RPO 15 min, RTO 4 h (pooled)	PITR, cross-region encrypted copy, quarterly restore drills
Security	Zero cross-tenant leakage; all privileged actions audited	RLS, MFA for staff, audit hash chain, pen-tests
Field reliability	Reader app fully usable offline for a full route-day	SQLite store, background sync, conflict policy
Accessibility & reach	Works on low-end Android and slow networks; SMS fallback for customers	Lightweight PWA, SMS/USSD channels (later)

8.1 Deployment variants

Variant	Who runs it	Data location	Commercial shape
SaaS pooled	404HubSpot	Shared cell, logically isolated	Per-connection subscription
SaaS dedicated	404HubSpot	Dedicated cell in chosen region	Subscription + dedicated infra fee
Sovereign / licensed	Client or their IT partner	Client environment	Licence + setup + support contract; IP stays with 404HubSpot unless sold separately

8.2 Open architecture questions

1. Does ZUCAR operate a water service itself (tenant) or only resell (partner)? Drives the partner-portal scope.
2. Will the first utility accept hosting outside Kenya, or must data be in-country? Drives region choice (MB-05, MB-07).
3. Is the utility’s existing meter and customer data clean enough to import, and in what format?
4. Does each tenant have its own Paybill/Till, or does the client expect a shared aggregator?
5. Which WASREB reports must be produced on day one?